

can you explain latest databricks concepts, lakehouse rt, genie ontology, rayden and so on in brief which was showcased in recent AI submit

Worked for 10s >

The recent **Databricks Data + AI Summit 2026** announcements show Databricks moving from a traditional analytics platform toward a unified platform for **real-time applications, enterprise agents, governed AI and autonomous data operations**.

First, a correction: the new engine is called **Reyden**, not Rayden.

1. Lakehouse//RT

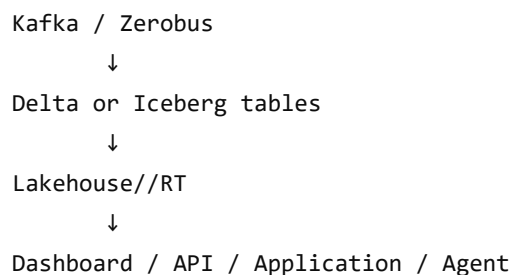
Lakehouse//RT is Databricks' new real-time analytical warehouse.

Previously, when applications required millisecond responses, data was usually copied from Delta Lake into systems such as Redis, ClickHouse, Elasticsearch, Pinot or Druid.

Lakehouse//RT aims to provide:

- Millisecond or sub-second SQL queries
- High query concurrency
- Real-time dashboards
- Operational analytics
- Application-serving APIs
- Observability and monitoring workloads
- Direct access to governed Delta and Iceberg data

The main architectural idea is:



This reduces the need for a separate serving database and the associated replication pipeline.  Databricks +2

Where it fits

For your high-throughput regulatory architecture, Lakehouse//RT could potentially serve:

- FCFS decision lookup
- Operational trade-status dashboards
- Submission SLA monitoring
- Real-time exception investigation
- Low-latency access to winner-selection results

However, it is primarily a **real-time analytical and serving engine**, not necessarily a replacement for a consensus mechanism such as Raft when strict distributed locking or global ordering is required.

2. Reyden engine

Reyden is the new compute engine underneath Lakehouse//RT.

Think of Databricks engines this way:

Engine	Primary purpose
Spark	Large-scale ETL, streaming and ML
Photon	Fast analytical SQL and data processing
Reyden	Millisecond, high-concurrency real-time queries

Reyden is optimized for workloads where many users, applications or agents simultaneously query recent data and expect immediate responses.

Conceptually:

Photon

Best for: warehouse analytics and large scans

Reyden

Best for: small, selective, repeated, low-latency queries



It operates over lakehouse-managed data, allowing organisations to avoid maintaining a separate low-latency copy solely for serving queries.  Databricks +1

3. Genie One

Genie One is the new simplified AI workspace for business users. It was previously known as **Databricks One**.

Instead of navigating catalogs, notebooks, SQL warehouses and dashboards, a business user gets one interface where they can:

- View dashboards
- Ask questions in natural language
- Analyse KPIs
- Search enterprise knowledge
- Invoke agents
- Take actions through connected applications

Example:

User:

"Why did regulatory submission failures increase today?"



Genie One:

1. Identifies the relevant KPI.
2. Queries submission and rejection tables.
3. Reviews dashboards and previous queries.
4. Explains the cause.
5. Links to the supporting data.

It is effectively intended to become an **AI coworker interface for business teams**.

 Databricks Docu... +1

4. Genie Ontology

Genie Ontology is one of the most important announcements.

It is an automatically generated and continuously updated map of:

- Business terms
- Entities
- Relationships
- KPIs
- Data assets
- Queries
- Dashboards
- Pipelines
- Documents
- Connected enterprise applications

For example, your physical data may contain:

```
trade_hdr.trd_id
submission_fact.first_arrival_ts
reg_response.status_cd
```



The ontology could learn that these correspond to:

```
Trade
First-arrival time
Regulatory submission status
```



It may also understand relationships such as:

```
Trade → has a submission
Trade → has a winning cloud
Submission → sent to a regulator
Submission → has an SLA status
```



This helps Genie understand that:

“How many trades missed the ten-second regulatory SLA?”

should be translated using the correct tables, timestamp definitions, filters and business rules.

The ontology is inferred from tables, queries, dashboards, pipelines and connected tools rather than being built entirely by hand. Permissions from the underlying sources continue to apply.  Databricks +2

Difference from a semantic layer

Semantic layer	Genie Ontology
Usually manually modelled	Automatically inferred and updated
Mainly metrics and dimensions	Entities, relationships, knowledge and metrics
Primarily used for BI	Used by BI, agents, search and coding assistants
Often limited to structured data	Can include structured and unstructured context

A governed semantic layer remains useful. The ontology adds a wider enterprise knowledge graph around it.

5. Genie Agents

Genie Agents are domain-specific conversational agents grounded in enterprise data.

A domain expert configures an agent with:

- Relevant datasets
- Sample questions
- Example SQL
- Business instructions
- Definitions
- Approved tools
- Access permissions

Example agents could include:

Regulatory Submission Agent
Fraud Investigation Agent
Finance Reconciliation Agent
Data Quality Agent
Customer Service Agent



A regulatory agent might answer:

“List trades for which AWS won FCFS, but the GCP submission reached the regulator first.”

Because it is grounded in governed data and ontology, it should produce more reliable answers than a generic chatbot.  Databricks Docu... +1

6. Genie Code

Genie Code is the agentic coding assistant for data engineers, analytics engineers and ML engineers.

It can help with:

- Creating notebooks
- Writing SQL and PySpark
- Building Lakeflow pipelines
- Debugging failed jobs
- Updating multiple assets
- Creating dashboards
- Analysing models

- Reviewing production systems

The new full-page command centre allows longer, multi-step tasks rather than only short code suggestions.

Example:

“Analyse this Oozie repository, convert the workflows into Databricks Asset Bundle YAML, create notebooks for Hive actions and report unresolved dependencies.”

Genie Code can work across notebooks, jobs, pipelines, dashboards, models and Unity Catalog assets. Databricks also announced scheduled tasks, allowing some work to be executed autonomously.  Databricks

7. Genie ZeroOps

Genie ZeroOps extends the agent concept from development into operations.

The intended direction is for agents to continuously:

- Detect pipeline failures
- Diagnose root causes
- Recommend fixes
- Optimise performance
- Monitor data quality
- Analyse cost anomalies
- Handle operational remediation

The conceptual flow is:

Observe → Diagnose → Recommend → Execute → Validate



For example:



```
graph TD; A[Streaming latency has increased] --> B[Agent detects Kafka backlog]; B --> C[Identifies skewed partition]; C --> D[Recommends repartitioning or scaling]; D --> E[Applies an approved change]; E --> F[Validates latency recovery];
```

It does not mean all operations should immediately run without human control. Approval policies, auditability and restricted tool permissions remain important.

8. Unity AI Gateway

Unity AI Gateway provides a central control layer for enterprise AI interactions.

It governs not just data assets, but also:

- Foundation models
- AI agents
- MCP servers
- Agent tools
- Skills
- Prompts and responses
- Token costs
- Runtime actions

Key functions include:

- Usage and cost tracking
- Spending limits
- Model routing
- Guardrails
- Access policies
- Agent monitoring
- End-to-end tracing
- Security investigation

Example:



```
graph TD; A[Application / Agent] --> B[ ];
```

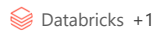
Unity AI Gateway

- └ Access control
- └ Prompt filtering
- └ Cost limit
- └ Model selection
- └ Tool permission
- └ Audit logging



OpenAI / Anthropic / Gemini / Databricks model

This is similar to an API gateway, but specifically for models, agents and AI tools.



9. Unity Catalog Glossary and Domains

Databricks is strengthening Unity Catalog as the business-context layer for both humans and agents.

Glossary

Stores governed business definitions such as:

First-arrival timestamp
Regulatory SLA
Winning cloud
Valid trade
Submission completion



Domains

Organise assets by business area:

Trading
Regulatory Reporting
Risk
Finance
Customer
Fraud



Together, they help agents interpret data consistently rather than independently inventing definitions.  Databricks

10. MCP integration

Databricks is embracing the **Model Context Protocol**, or MCP.

MCP provides a standard mechanism through which an AI agent can access enterprise tools and data.

Databricks can expose Genie as an MCP server. An external agent can then send a business question to Genie, which:

1. Interprets the request through Genie Ontology.
2. Queries governed enterprise data.
3. Returns a grounded result.
4. Provides links to supporting Databricks assets.

This means a coding assistant, internal chatbot or external agent platform can use Genie as an enterprise data tool without independently rebuilding all the semantic logic.

 Databricks Docu...

11. Unity Catalog for cross-cloud governance

Unity Catalog is moving toward one governance model covering:

- Multiple Databricks workspaces
- Multiple regions
- Multiple clouds
- Delta and Iceberg
- Models and agents
- MCP tools
- External engines

The objective is:

AWS Databricks 
Azure Databricks  One catalog and policy model
GCP Databricks 



This is especially relevant for your AWS–GCP active-active design, although the data plane, replication behaviour and regional failure characteristics still need to be designed separately.

12. Managed Iceberg and open interoperability

Databricks continues to make the platform more open through greater Apache Iceberg support.

Managed Iceberg allows organisations to:

- Create and manage Iceberg tables in Unity Catalog
- Read the same data from different engines
- Maintain governance centrally
- Reduce proprietary table-format lock-in

The broader vision is:

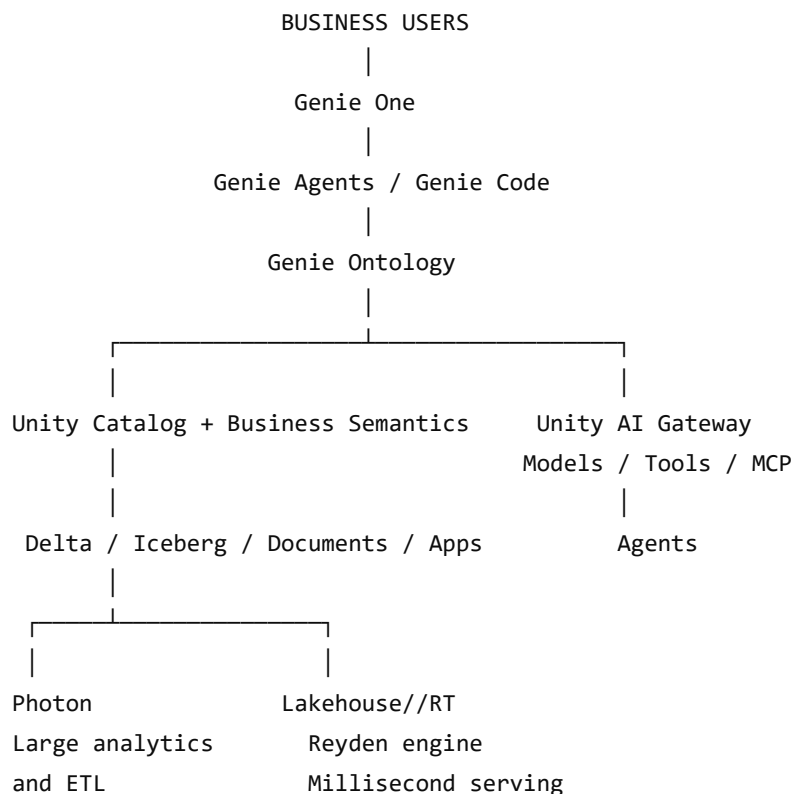
One governed dataset

- ├─ Databricks
- ├─ Spark
- ├─ Trino
- ├─ Snowflake-compatible integrations
- └─ Other Iceberg engines



Managed Iceberg became generally available around the Summit period.  Databricks

How the new platform fits together



The biggest architectural shift

Earlier Databricks was mainly viewed as:

Data engineering + lakehouse analytics + machine learning.

The emerging model is:

One governed enterprise context layer powering data pipelines, real-time applications, BI users, developers and autonomous AI agents.

The four announcements I would prioritise are:

1. **Lakehouse//RT and Reyden** — real-time serving directly from the lakehouse.
2. **Genie Ontology** — automatic enterprise context and business knowledge.
3. **Genie One and Genie Agents** — conversational analysis moving toward business actions.
4. **Unity AI Gateway** — central governance for models, agents, tools and AI costs.

        Sources

agent bricks is still there?

Agent Bricks is still there and remains Databricks' main platform for building and optimizing enterprise AI agents. Databricks specifically expanded it at **Data + AI Summit 2026**.  Databricks +1

The relationship is roughly:

Agent Bricks

- Build and customize agents
- Connect agents to data and tools
- Generate synthetic training/evaluation data
- Evaluate quality
- Optimize model, prompts, quality and cost
- Deploy agents
- Govern them through Unity Catalog / AI Gateway



What changed in 2026

Agent Bricks is becoming a **broadly governed enterprise-agent platform**, rather than only a no-code agent builder. It now covers agents connected to business data, models, workflows and MCP tools.  Databricks +1

The newer concepts complement it:

Concept

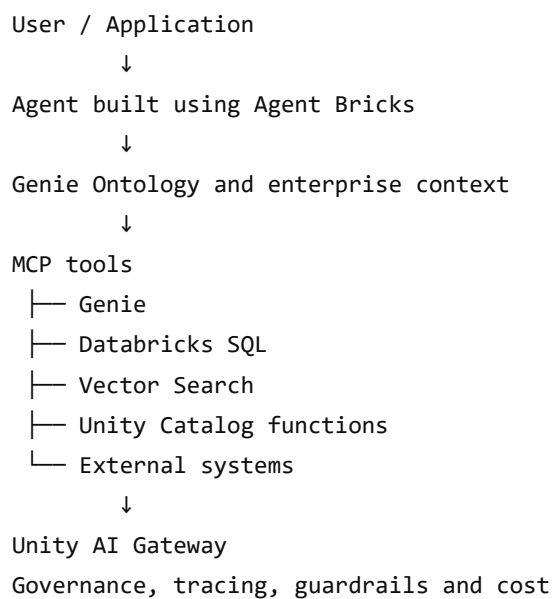
Role

Agent Bricks

Builds, evaluates and optimizes agent systems

Concept	Role
Genie Agents	Ready-made, business-data-focused conversational agents
Genie Ontology	Gives agents business terminology and relationships
Unity AI Gateway	Governs models, agents, tools, cost and usage
MCP servers	Allow agents to access Genie, SQL, search and functions
Databricks Apps	Hosts custom agent applications and user interfaces
Genie Code	Coding and engineering agent
ZeroOps	Operational agents for monitoring and remediation

A simple architecture would be:



One important distinction:

- **Agent Bricks** is the overall agent development and optimization capability.
- **Genie Agent** is a particular business intelligence/data-questioning agent.
- **Agent Framework / Databricks Apps** is used when developers need complete code-level control.

Some older individual Agent Bricks experiences, such as the original Information Extraction and Custom LLM interfaces, are now marked **legacy**, but this does not mean Agent Bricks itself is discontinued. Databricks is replacing specific older workflows with newer versions while retaining and expanding the Agent Bricks brand.  Databricks Docu... +1

I am working in banking domain. can you give few use cases in banking sector using agent bricks which are still open or latest ask in real world

Worked for 12s >

Yes. In banking, the strongest Agent Bricks opportunities are not generic chatbots. They are **investigation and operations agents** that assemble evidence, follow bank policies, interact with multiple systems and leave a complete audit trail.

Agent Bricks is well suited because Databricks positions it for building, evaluating, optimizing and governing production agents over enterprise data. Current financial-services messaging specifically targets front-, middle- and back-office workflows.

 Databricks +2

Best open banking use cases

1. Mule-account investigation agent

Problem

Banks can detect suspicious transactions, but investigators still spend significant time manually connecting:

- Customer and KYC records
- Beneficiary relationships
- Device and IP information
- UPI, IMPS and account transactions
- Shared addresses and phone numbers
- Previous alerts and complaints
- Cash-out patterns

RBI specifically asks regulated entities to monitor unusual activity involving non-KYC-compliant and money-mule accounts.  Reserv...

Agent workflow

Transaction alert



Collect customer, device and transaction history



Construct beneficiary and device network



```
↓
Identify mule-account indicators
↓
Generate evidence-backed case summary
↓
Recommend hold, enhanced review or closure
↓
Human investigator approves action
```

Why this remains open

Traditional AML systems generate alerts, but they do not perform a complete, contextual investigation. The difficult part is combining structured transactions with unstructured KYC documents, call notes and previous investigation reports.

Agent Bricks components

- Knowledge Assistant for policies and historical cases
- Custom agent for investigation orchestration
- Genie or SQL tools for transaction analysis
- Graph-analysis functions for account relationships
- Document Intelligence for KYC evidence
- MLflow evaluation using confirmed mule cases

Suggested POC KPI

Reduce average investigation time from, for example, 45 minutes to under 15 minutes without lowering investigator-confirmed precision.

2. AML alert disposition agent

This is broader than mule-account detection.

Problem

AML investigators must determine whether an alert represents:

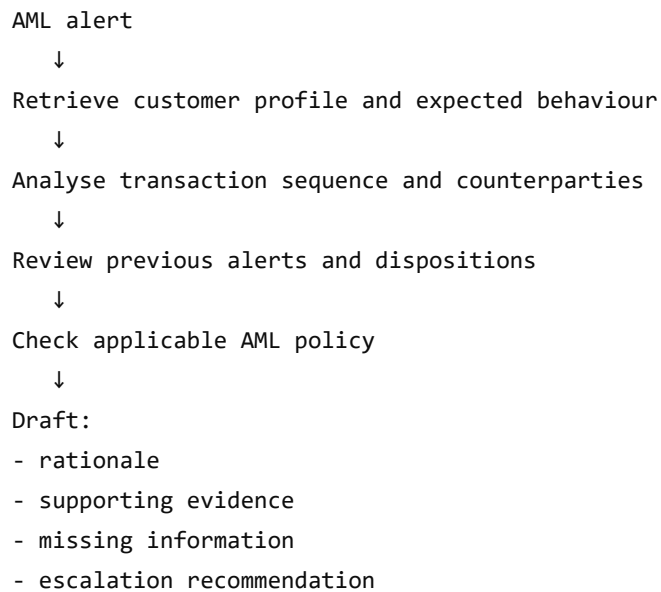
- Genuine suspicious behaviour
- Expected customer activity
- Duplicate alerts
- Previously investigated activity
- A scenario requiring escalation or STR preparation

The EBA continues to highlight evolving AML risks from fintech, crypto assets, AI-driven fraud, RegTech weaknesses and sanctions compliance. It has also warned that careless

adoption of compliance technology can create additional money-laundering risk.

 European Ba... +1

Agent workflow



```
graph TD; A[AML alert] --> B[Retrieve customer profile and expected behaviour]; B --> C[Analyse transaction sequence and counterparties]; C --> D[Review previous alerts and dispositions]; D --> E[Check applicable AML policy]; E --> F["Draft:  
- rationale  
- supporting evidence  
- missing information  
- escalation recommendation"];
```

AML alert

↓

Retrieve customer profile and expected behaviour

↓

Analyse transaction sequence and counterparties

↓

Review previous alerts and dispositions

↓

Check applicable AML policy

↓

Draft:

- rationale
- supporting evidence
- missing information
- escalation recommendation

The agent should **draft a disposition**, not independently close a high-risk alert.

Latest real-world requirement

Banks increasingly want an agent that understands why an alert fired, reconstructs the transaction story, cites every source and explains why a recommended disposition complies with policy.

3. Sanctions investigation agent

Problem

A sanctions screening match often requires manual comparison of:

- Customer name
- Aliases
- Date of birth
- Nationality
- Address
- Ownership structure
- Vessel, company or counterparty details
- Adverse media
- Applicable sanctions list

Simple fuzzy-name matching causes excessive false positives.

Agent workflow

```
graph TD; A[Possible sanctions match] --> B[Collect entity and beneficial-owner data]; B --> C[Compare against sanctions-list attributes]; C --> D[Search internal transactions and relationships]; D --> E[Explain matching and conflicting attributes]; E --> F[Recommend false positive / enhanced review / escalation];
```

Why it is attractive

The agent does not replace the screening engine. It handles the expensive **post-match investigation**.

Open challenge

The agent must preserve the exact sanctions-list version, retrieval date, evidence and reasoning used for the decision. This is an excellent use case for Unity Catalog lineage, controlled tools and MLflow traces.

4. Fraud complaint-to-investigation agent

Problem

When a customer reports an unauthorised payment, the bank must quickly determine:

- What transactions are affected
- Whether credentials or devices changed
- Whether the beneficiary is connected to other fraud cases
- What customer communication occurred
- Which accounts may require a hold
- Whether regulatory or law-enforcement reporting applies

RBI's fraud-risk directions require reporting and investigation processes, with specific expectations around reporting fraud incidents to law-enforcement agencies.

 Reserve Bank of ... +1

Agent workflow



```
graph TD; A[Customer complaint] --> B[Extract transaction IDs and complaint details]; B --> C[Retrieve authentication, device and payment events]; C --> D[Identify connected beneficiary accounts]; D --> E[Prepare fraud timeline]; E --> F[Recommend containment steps]; F --> G[Draft investigation and customer-response notes];
```

Important differentiation

Most current fraud models produce a score. The open requirement is an agent that converts alerts, logs, complaints and policy into an **actionable investigation package**.

5. Regulatory reporting reconciliation agent

This is especially relevant to your regulatory-submission background.

Problem

Banks submit reports to regulators from multiple source systems. Common issues include:

- Source-to-report mismatches
- Missing records
- Late-arriving transactions
- Incorrect classifications
- Duplicate submissions
- Differences between finance, risk and regulatory numbers
- Unexplained restatements

Agent workflow



```
graph TD; A[Reporting discrepancy] --> B[Identify affected report cells]; B --> C[Trace values to source tables]; C --> D[Compare current and previous submissions]; D --> E[Check transformation rules];
```

↓
Locate failed or delayed pipelines
↓
Draft reconciliation explanation
↓
Create remediation tasks

Why it remains open

Data lineage can show where a number came from, but it normally does not explain:

“Why did this value change by ₹280 crore from yesterday, and was the change expected?”

An agent can combine lineage, business rules, pipeline history, source data and submission acknowledgements.

Strong POC

Build an agent for one report, such as liquidity, capital adequacy, large exposure or transaction reporting, and measure:

- Reconciliation time
- Number of automatically explained breaks
- False explanation rate
- Time required to prepare regulator evidence

6. Credit-memo and covenant-monitoring agent

Problem

Relationship managers and credit analysts review:

- Financial statements
- Bank statements
- Credit-bureau reports
- Company filings
- Existing exposure
- Covenants
- Industry conditions
- News and legal events

Much of this process remains manual.

Agent workflow

Borrower review



Extract financial and covenant information



Calculate ratios and trends



Compare against sanctioned terms



Identify adverse changes



Draft credit-review memo



Highlight missing evidence



Open and valuable variation

A **continuous covenant-surveillance agent** watches borrower information and alerts the analyst when evidence suggests a possible breach.

It should not autonomously approve or decline credit. Credit decisions can create significant model, fairness and explainability risks; regulators increasingly focus on governance of high-impact AI used in banking. 📌 Bank... +2

7. Early-warning signal investigation agent

Banks already have early-warning models. The missing layer is often explaining and operationalising the signal.

Inputs

- Missed or delayed payments
- Declining balances
- Cheque returns
- Unusual fund transfers
- GST or sales changes
- Credit-bureau changes
- Covenant breaches
- Industry stress
- Customer-service interactions

Agent output

Risk increased because:

1. Monthly credits fell by 38%.
2. Three EMI payments were delayed.



3. The borrower transferred funds to a related entity.
4. The relevant industry indicator deteriorated.
5. Two covenants require review.

It can then recommend:

- Relationship-manager review
- Updated financial documents
- Limit reassessment
- Watch-list referral
- Collections engagement

The difficult open problem is converting hundreds of signals into a defensible narrative rather than simply producing another probability score.

8. Payment-operations exception agent

Problem

Payment failures and exceptions are distributed across:

- Core banking
- Payment switch
- UPI or card network
- Fraud system
- Ledger
- Reconciliation platform
- Middleware
- Customer-support system

EBA operational-resilience reporting indicates that payment services and IT systems remain major sources of reported incidents.  European Banki... +1

Agent workflow

```
Payment failed or is stuck
↓
Correlate event across systems
↓
Identify last successful processing stage
↓
Check known incident and runbook
↓
Determine financial impact
↓
Recommend retry, reversal or manual repair
```





Draft customer and operations communication

This is a strong Agent Bricks use case because it combines SQL queries, logs, runbooks, ticketing and controlled operational actions.

9. Operational-resilience and incident commander agent

Problem

During an outage, teams lose time identifying:

- Which banking services are affected
- Whether customer funds are at risk
- Which upstream or downstream systems are involved
- Regulatory notification requirements
- Recovery status
- Whether SLA or RTO thresholds have been crossed

Agent workflow

Incident detected



Correlate logs, metrics and pipeline failures



Map affected services and customers



Retrieve recovery runbooks



Create incident timeline



Draft management and regulatory updates



Track recovery actions



This is one of the most current enterprise-agent asks because operational risk remains elevated and banks are under pressure to strengthen digital operational resilience.

 European Banki... +2

10. Model-risk validation agent

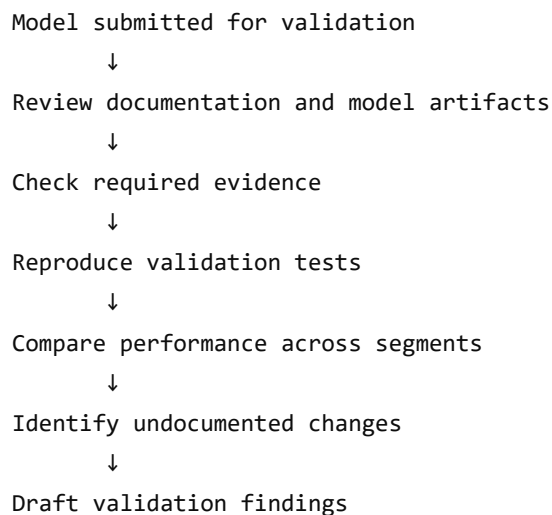
Problem

Banks may have thousands of credit, fraud, AML, market-risk and GenAI models. Model validators must inspect:

- Model documentation
- Training data
- Assumptions
- Performance
- Drift
- Overrides
- Validation findings
- Regulatory requirements
- Production changes

Databricks identifies model-risk management as a significant banking priority in 2026, including the need to unify evidence, controls and governance.  Databricks

Agent workflow



```
graph TD; A[Model submitted for validation] --> B[Review documentation and model artifacts]; B --> C[Check required evidence]; C --> D[Reproduce validation tests]; D --> E[Compare performance across segments]; E --> F[Identify undocumented changes]; F --> G[Draft validation findings];
```

Particularly new requirement

Create an **agent-validation agent** that tests other banking agents for:

- Hallucination
- Unsupported conclusions
- Policy violations
- Bias
- Sensitive-data leakage
- Unsafe tool calls
- Excessive autonomy
- Cost and latency

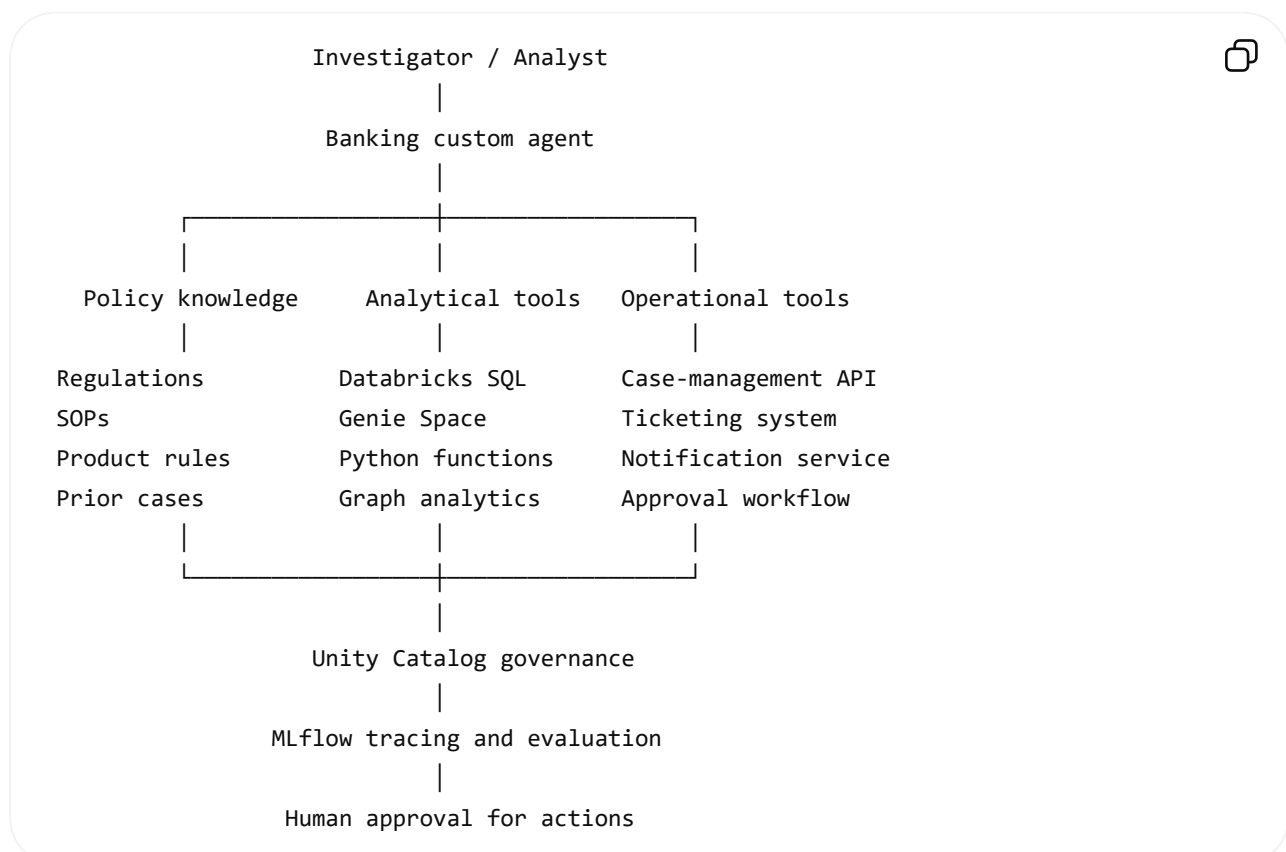
Managed MLflow provides agent tracing, evaluation and AI-judge capabilities that can support this control layer.  Databricks +1

My top four recommendations for a banking POC

Priority	Use case	Value	Complexity
1	Regulatory reporting reconciliation	Very high	Medium
2	Mule-account investigation	Very high	High
3	Payment exception investigation	High	Medium
4	Model-risk validation agent	High	Medium-high

For your background, **regulatory reporting reconciliation** would be the strongest starting point. It aligns with streaming, SLA, Databricks, data lineage and regulator-submission experience.

Example Agent Bricks architecture



What banks should avoid initially

Do not begin with an agent that autonomously:

- Rejects a loan
- Blocks a customer permanently
- Files an STR without investigator review
- Freezes accounts solely from LLM reasoning
- Changes regulatory submissions
- Executes large financial transactions
- Communicates a final fraud decision to customers

The initial operating model should be:

```
Agent investigates
  ↓
Agent explains and recommends
  ↓
Human approves
  ↓
Deterministic workflow executes
  ↓
Everything is logged and evaluated
```



This human-controlled design better addresses the explainability, model-risk, operational-risk and governance concerns currently raised by banking regulators.  Bank for Interna... +3

     ...    Sources